Name: Jacinto Jimenez
Class: CS 79C Section 1753
Instructor: Vicky Seno
Date: October 27, 2026

AWS Architecture Portfolio Project

Project: Incident Handoff Tracker | AWS Region: us-west-2 (Oregon)

DRAFT - planned architecture. AWS deployment, Pricing Calculator totals, screenshots, and final deletion have not yet been verified.

Executive Summary

This project addresses the problem of fragmented technical-support handoff notes by proposing a small, reproducible incident-tracking API for fictional training records. The architecture uses Amazon API Gateway, AWS Lambda, Amazon DynamoDB, Amazon S3, and Amazon CloudWatch, deployed together through AWS CloudFormation. Its purpose is to demonstrate an end-to-end cloud workflow, security and cost trade-offs, and evidence-based operating procedures rather than to store real company data.

Business Problem and Users

During a support handoff, useful context may be scattered across conversations, temporary notes, and tickets. A demo incident record containing an ID, title, severity, short summary, creation time, and author shows how a standardized schema can make a handoff retrievable without relying on any real proprietary data. The expected audience is a support analyst or reviewer examining a fictional incident; the system is a portfolio demonstration, not a production support platform.

What the Application Does

- A public health-check endpoint returns a small JSON document with the project name, region, and my name so a reviewer can verify that the deployment is reachable.
- An AWS IAM-protected POST route validates a fictional incident, assigns a UUID, inserts the record into DynamoDB, and writes a JSON copy to private S3.
- An IAM-protected GET route retrieves an incident by its UUID, while CloudWatch stores execution logs and watches the Lambda Errors metric.

The public endpoint exposes no incident records. Incident read/write operations require AWS Signature Version 4 authorization and therefore are not intended as anonymous browser forms.

Architecture and Service Selection

Figure 1 in Appendix A shows the intended request and data flows. All resources are managed by one CloudFormation template: API routes forward to a Lambda proxy integration; the function uses a scoped IAM role to write DynamoDB records and private S3 JSON objects; CloudWatch records errors. This meets the project requirement for four or more integrated course services without a continuously running virtual machine.

Service selected	Role and rejected alternative
API Gateway HTTP API	Provides HTTPS routes and IAM authorization; preferred over an always-on Application Load Balancer for a very low-traffic demo.
AWS Lambda (Python)	Validates requests and orchestrates persistence; preferred over EC2 because there is no server to patch or pay for while idle.
DynamoDB on-demand	Stores incidentId-keyed records with automatic capacity; preferred over RDS for a simple key-value workload.
S3 Standard	Stores private per-record JSON snapshots; preferred over EC2 local disk for inexpensive object storage.
CloudWatch Logs + Alarm	Captures function execution and observes the Errors metric; preferred over deploying a separate monitoring server.
CloudFormation	Versioned IaC creates the system as one stack; preferred over Terraform here for direct AWS Console upload with no local providers.

Key Infrastructure as Code Choices

The template defines an S3 public-access block and SSE-S3 encryption; DynamoDB with pay-per-request billing and AWS-owned default encryption; a Python 3.12 Lambda function with 128 MB memory; API Gateway stage throttling at two requests/second with a burst of five; seven-day archive/log retention; and a CloudWatch alarm for one or more Lambda errors in five minutes. IAM grants only DynamoDB PutItem/GetItem, S3 PutObject within the project bucket path, and the log permissions needed by the function.

The complete executable CloudFormation source is committed as template.yaml in the repository. The Python handler is also present as src/handler.py so the application can be code-reviewed independently.

Implementation, Testing, and Challenges

The reproducible deployment procedure is to upload template.yaml using the CloudFormation console in us-west-2, enter the stack name incident-handoff-portfolio, acknowledge IAM resource creation, and monitor stack Events until CREATE_COMPLETE. The Outputs section then provides the API health URL, DynamoDB table name, S3 bucket name, and Lambda function name. A public /health GET should return a JSON payload with my name.

The data-path test creates a fictitious incident through a Lambda console event or an IAM-signed API request, records the returned incidentId, verifies that DynamoDB contains the corresponding item, and checks for incidents/<id>.json in S3. A second test retrieves the same record. Four local unit tests using mocked clients have passed; these are code checks only, not evidence that AWS deployment has already occurred.

The main design trade-off is that a successful DynamoDB write is followed by an S3 upload as two separate operations. If the second operation fails, an incident may exist in the table without a snapshot. A production design would use DynamoDB Streams, SQS, or an event-driven replay workflow with retries and a dead-letter queue instead of relying on this synchronous dual write.

Security

The three design controls are authentication, least privilege, and encryption. API Gateway uses AWS_IAM authorization for POST and incident GET routes, Lambda assumes an IAM execution role instead of storing access keys, and access to the DynamoDB table/S3 object path is restricted to the named resources. The HTTPS endpoint encrypts transit; DynamoDB default encryption at rest and S3 SSE-S3 protect stored data. The health endpoint is intentionally public and returns only demo metadata; input length limits and basic throttling reduce simple misuse but do not replace WAF, user identity management, or a full threat model.

Reliability and Recovery

The selected compute, database, gateway, and object-storage services are managed regional services that avoid a single personally managed EC2 server. The design still has a single-Region failure domain and no proven cross-Region disaster recovery. Proposed but unverified targets are RTO within four hours and RPO within one hour; an S3 archive can assist manual recovery only for successfully archived records. DynamoDB point-in-time recovery and a tested restoration procedure would be required before asserting those objectives are met.

Performance and Cost

Lambda and DynamoDB on-demand capacity can absorb changes in request volume without scheduling EC2 instances. The API stage deliberately throttles traffic for this course demo. Potential performance bottlenecks include Lambda cold starts, writing each incident sequentially to DynamoDB then S3, and throttling once the configured limit is reached. A load test is planned but not claimed as complete.

Cost inputs for an official AWS Pricing Calculator estimate are: 1,000 HTTP API requests/month, 1,000 Lambda invocations at 128 MB and 150 ms average, 300 DynamoDB writes plus 700 reads, approximately 2 MB database storage, 300 small S3 PUTs, seven-day logs, and one CloudWatch standard-resolution alarm. The final report must include the calculator-generated monthly total, date, and screenshot; it is not yet available. Optimization 1 is eliminating EC2/ALB/NAT costs with managed serverless services; optimization 2 is pay-per-request capacity plus short retention for low-volume demo records.

Well-Architected Self-Review

Pillar	What works well	What I would improve
Security	IAM-protected incident routes; least-privilege service role; encrypted private storage.	Add Cognito/JWT end-user identity, WAF, key-policy review, and secret scanning.
Reliability	Managed regional services and JSON snapshots for successful writes.	Add transactional event flow, retries, PITR, tested restore, and Region resilience.
Cost Optimization	No idle EC2, pay-per-request database, 128 MB Lambda, seven-day retention.	Verify a calculator and measured AWS bills; tune logs and alarm charges.

What I Would Do Differently

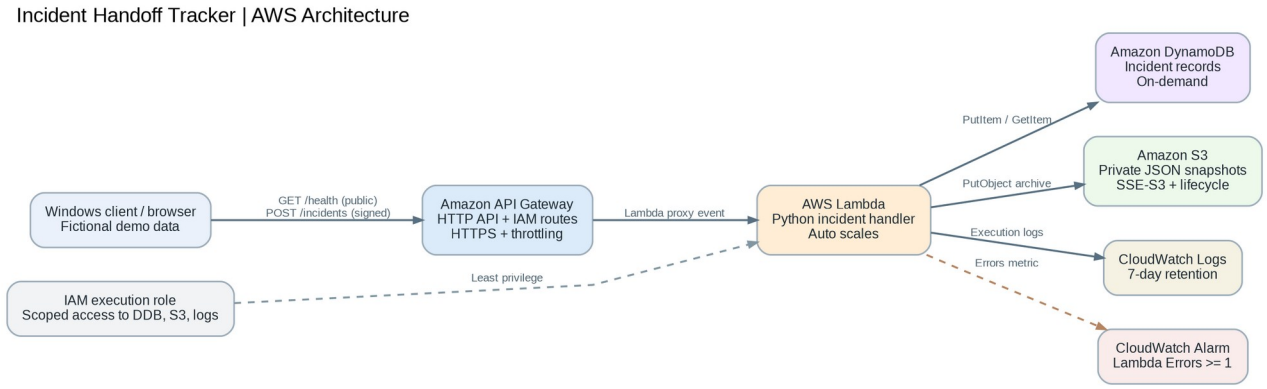
With more time, I would replace synchronous DynamoDB-plus-S3 writes with DynamoDB Streams and an asynchronous archiving Lambda. This would add delivery status, explicit retry behavior, and a recovery path if S3 is temporarily unavailable. I would also add authenticated user-facing views, measured performance tests, DynamoDB PITR, a Region-level recovery plan, and a signed-off budget estimate.

Evidence and Cleanup Verification

Appendix B is an evidence checklist, not a substitute for live AWS screenshots. Before submission, insert screenshots of the deployed URL, CloudFormation CREATE_COMPLETE, API or Lambda tests, matching DynamoDB/S3 records, CloudWatch alarm, and the AWS Pricing Calculator result. After capturing them, empty the S3 bucket and delete the CloudFormation stack; only mark cleanup complete after AWS confirms it.

Appendix A - Architecture Diagram

Figure 1. Proposed data flow and monitoring. The diagram is also available as diagram/architecture.svg in the GitHub repository.



The operational and security controls are described in the four-page narrative above. This appendix is outside the report narrative page limit.

Appendix B - Live AWS Evidence To Be Added

This appendix deliberately contains no fabricated console screenshots. Replace this checklist with actual AWS console and application evidence after deployment.

- CloudFormation stack in CREATE_COMPLETE status (Oregon region).
- Browser showing the actual public /health endpoint and my name.
- Real POST response (statusCode 201) with a unique fictional incidentId.
- DynamoDB record and S3 JSON object with the same incidentId.
- CloudWatch Lambda execution log and Errors alarm.
- AWS Pricing Calculator date, use assumptions, and monthly total.
- CloudFormation DELETE_COMPLETE or confirmed absence of active resources after emptying S3.

Reference Documentation

AWS CloudFormation - Lambda Function Code: https://docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/aws-properties-lambda-function-code.html

AWS API Gateway HTTP API pricing: https://aws.amazon.com/api-gateway/pricing/

AWS Lambda pricing: https://aws.amazon.com/lambda/pricing/

Amazon DynamoDB pricing: https://aws.amazon.com/dynamodb/pricing/

AWS Pricing Calculator: https://aws.amazon.com/aws-cost-management/aws-pricing-calculator/

AWS Well-Architected Framework: https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html